CNuvS Ex. 6 - Jan Lindauer, Paul Feidieker

Task 4

(a) 1. Segmentation, Addressing, Multiplexing
2. Connection Control
[1, p. 4]

(b) The component 130.83.47.181 is the IP address representing the receiving host on the network, which a reverse DNS lookup identifies as a server belonging to TU Darmstadt (Hostname: cms-sip02.hrz.tu-darmstadt.de.). The component 443 is the port number representing the specific receiving service, which stands for HTTPS. [1, p. 5], [2]

(c) The three-way handshake only works if each of the handshake messages are properly recieved in time by the respective party. Since communication channels are propabilistic channels, we can't guarantee that each message arrives correctly. From there several problems arise. The ACK-message, the second DR-message or even the first DR-message after a resend can fail to arrive. Each of these problems is solved by setting optimistic timeouts that trigger a retransmission of the pontentially failed message. [1, p. 7]

(d) The major limitation of Alternating Bit Protocol is that it can only handle one packet at a time, which means that the sender has to wait for an acknowledgment for each packet before sending the next one. This leads to inefficient use of network resources, especially in high-latency networks, as the sender spends a lot of time waiting for acknowledgments instead of transmitting data.
This becomes severe if the distance between the sender and receiver is large, leading to increased latency due to the longer propagation time and reduced throughput. [1, p. 8, 9]

Task 5

(a) The IP address is a defining part of the IP protocol on the network layer, while the networking port is a responsibility of the transport layer. The OSI model advocates for clear abstractions that encapsulate finer grain technicalities away from higher layers. Since the internet is older than the OSI model it still combines IP addresses and ports as socket identifiers. This has some major upsides like defining unique indefications for application endpoints and also enables are more efficient multiplexing between concurrently running network applications.
[1, Part 1, p. 18, 19]

(b) Using PIDs would be problematic because processes can are created and killed dynamically, which means that the PID associated with a particular service may change over time. A device may also be rebooted which usually changes the PID. In these cases, everytime the PID changes, the new PID would need to be announeced to the network.
Instead, abstract service addresses called Protocol Ports are used.
[1, Part 1, p. 17]

(c) If both requests use UDP, 1 socket is needed on the server side, since UDP is connectionless and does not require a dedicated connection for each sender. The server can simply listen on the same port for incoming UDP packets from both clients.

If both requests use TCP, 2 sockets are needed on the server side, since TCP is connection-oriented and requires a dedicated connection for each sender. The server would need to create a seperate socket for each client to handle the TCP connection.
[1, Part 1, p. 9]

(d) The default port range for registered ports supported by Windows 10 is from 1024 to 49151. [3]
The default port range for ephemeral ports supported by Windows 10 is from 49152 to 65535. [3]

Task 6

(a) Packet size in bits:

$$L = 13 \times 1024 \times 8 \text{ bit} = 106\,496 \text{ bit}$$

Data rate in bit/s:

$$R = 35 \times 2^{30} \text{ bit/s} = 37\,580\,963\,840 \text{ bit/s}$$

Total data in bits:

$$D = 3.7 \times 1024 \times 1024 \times 8 \text{ bit} = 31\,063\,040 \text{ bit}$$

Transmission time per packet:

$$t_{\text{tx}} = \frac{L}{R} = \frac{106\,496}{37\,580\,963\,840} \approx 2.8338 \times 10^{-3} \text{ ms} = 2.8338\mu\text{s}$$

Number of Packets:

$$N = \left\lceil \frac{D}{L} \right\rceil = \left\lceil \frac{31\,063\,040}{106\,496} \right\rceil = \lceil 291.68... \rceil = 292 \text{ packets}$$

In the alternating bit protocol, the sender must wait for the ACK before sending the next packet. One full round-trip cycle takes:

$$\begin{aligned} t_{\text{cycle}} &= t_{\text{tx}}(\text{packet}) + t_{\text{prop}} + t_{\text{tx}}(\text{ACK}) + t_{\text{prop}} \\ &= 2 \cdot t_{\text{tx}} + 2 \cdot t_{\text{prop}} \\ &= 2 \times 2.8338\mu\text{s} + 2 \times 84 \text{ ms} \\ &\approx 0.0057 \text{ ms} + 168 \text{ ms} \\ &\approx 168.0057 \text{ ms} \end{aligned}$$

For the last packet we do not need to wait for the ACK to return:

$$\begin{aligned} T_{\text{total}} &= (N-1) \cdot t_{\text{cycle}} + t_{\text{tx}}(\text{packet}) + t_{\text{prop}} \\ &= 291 \times 168.0057 \text{ ms} + 2.8338\mu\text{s} + 84 \text{ ms} \\ &= 48\,889.6 \text{ ms} + 84.0028 \text{ ms} \\ T_{\text{total}} &\approx 48\,973.6 \text{ ms} \approx \mathbf{48.9736 \text{ s}} \end{aligned}$$

(b) The sender transmits all packets back-to-back without waiting for individual ACKs. Only one ACK is sent at the very end:

$$\begin{aligned} T_{\text{total}} &= N \cdot t_{\text{tx}}(\text{packet}) + t_{\text{prop}} + t_{\text{tx}}(\text{ACK}) + t_{\text{prop}} \\ &= (N+1) \cdot t_{\text{tx}} + 2 \cdot t_{\text{prop}} \\ &= 293 \times 2.8338\mu\text{s} + 2 \times 84 \text{ ms} \\ &= 830.3\mu\text{s} + 168 \text{ ms} \end{aligned}$$

$$T_{\text{total}} \approx 168.8303 \text{ ms} \approx \mathbf{0.1688 \text{ s}}$$

Sources

[1] Prof. Dr. Zsolt Istvan, “Computer Networks & Distributed Systems Chapter 4: Transport Layer.” 2026.

[2] “DNS Checker.” [Online]. Available: https://dnschecker.org/

[3] “Highportrange Risiko.” [Online]. Available: https://www.msxfaq.de/netzwerk/grundlagen/highportrangerisiko.html